

Large Language Models (LLMs) Cheat Sheet

22 tables · 147 concepts · Updated 2026-07-16 · [cheatgrid.com](#)

Large Language Models are **transformer-based neural networks** trained on massive text datasets to generate, understand, and manipulate human language at scale. At their core, LLMs use **self-attention mechanisms** to capture contextual relationships between tokens, enabling them to perform tasks ranging from translation and summarization to code generation and complex reasoning. The field has evolved rapidly—from foundational **pre-training** on trillions of tokens, through specialized **fine-tuning** and **alignment** techniques, to sophisticated **reasoning models** trained with reinforcement learning from verifiable rewards. A key insight: LLMs don't simply memorize text—their emergent abilities to **reason in-context** and solve unseen problems arise from the interplay of architecture, scale, and post-training dynamics, with 2025–2026 marking a shift toward agentic, multimodal, and long-context systems.

CONTENTS

Core Transformer Architecture	2
Positional Encoding Variants	2
Model Architecture Variants	3
Tokenization Algorithms	3
Pre-training Objectives	4
Fine-Tuning Techniques	4
Alignment and RLHF	5
Inference Optimization Techniques	6
Sampling and Decoding Strategies	7
Training Optimizations	7
Distributed Training Strategies	8
Model Compression Techniques	9
Context Window and Long-Context Techniques	9
Prompt Engineering Techniques	10
Emergent Capabilities and Scaling	10
Activation Functions in Transformers	11
Normalization Techniques	11
Evaluation Metrics and Benchmarks	12
Attention Mechanism Optimizations	13
Advanced Training Techniques	13
Model Merging Techniques	14
LLM Agent Concepts	14
References	15

TABLE 1

Core Transformer Architecture

Every modern LLM is built from this handful of components, stacked dozens of times over. Self-attention is the heart of it, letting each token weigh every other token, and the rest exist to make that work at depth and scale. Feed-forward layers add non-linear capacity, residual connections and normalization keep gradients flowing through hundreds of layers, and positional encoding restores the word-order information that attention alone throws away. Master these and you understand the skeleton shared by GPT, LLaMA, and Qwen alike.

Component	Example	Description
Self-attention	<code>scores = Q @ K.T / sqrt(d_k)</code>	- Computes query-key-value relationships where each token attends to all positions - forms the foundation of transformer parallelization by replacing recurrence.
Multi-head attention	<code>heads = 8</code> <code>Q, K, V = Linear(x, d_k)</code>	- Splits attention into multiple parallel heads learning different representation subspaces - each head computes scaled dot-product attention independently, then concatenates results.
Causal (masked) attention	<code>mask = torch.triu(ones) * -inf</code>	- Prevents tokens from attending to future positions via upper-triangular mask - critical for autoregressive generation in decoder-only models like GPT.
Cross-attention	<code>encoder_output → decoder</code>	- Used in encoder-decoder models where decoder queries attend to encoder keys/values - enables translation and seq2seq by bridging input-output representations.
Feed-forward network (FFN)	<code>FFN(x) = ReLU(xW1 + b1)W2</code>	- Two-layer MLP applied position-wise after attention - typically expands dimension 4× then projects back, providing non-linearity and feature transformation.
Residual connections	<code>output = x + SubLayer(x)</code>	- Adds input directly to sublayer output enabling gradient flow through deep networks - prevents vanishing gradients and allows training of 100+ layer transformers.
Layer normalization	<code>LN(x) = γ(x - μ) / σ + β</code>	- Normalizes activations across feature dimension for each token - stabilizes training and enables higher learning rates.
Positional encoding	<code>PE = sin(pos/10000^(2i/d))</code>	- Injects order information into token embeddings - the transformer is permutation-invariant without explicit position signals.
QK-Norm	<code>q = RMSNorm(q); k = RMSNorm(k)</code>	- Applies RMS normalization to query and key vectors before dot-product attention - stabilizes training of very large models and is used in Qwen3, Trinity Large.

TABLE 2

Positional Encoding Variants

Since attention treats a sequence as an unordered bag of tokens, a model needs some way to know which word came first, and the choice of how to inject that order has outsized consequences for how far a model can extrapolate beyond its training length. These variants trace the field's evolution from the original fixed sinusoids and learned absolute embeddings toward relative schemes like RoPE and ALiBi that generalize to longer contexts, and even NoPE, which drops explicit position signals entirely.

Method	Example	Description
RoPE (Rotary Position Embedding)	<code>rotate(q) @ rotate(k).T</code>	- Applies rotation matrices to query and key vectors so their dot product encodes relative position - used in LLaMA, GPT-NeoX; extrapolates well but long context still needs RoPE scaling (NTK, YaRN).
Learned absolute	<code>pos_emb = Embedding(max_len, d_model)</code>	- Trainable position embeddings learned during training - used in BERT and GPT-2 limited to max_len seen during training, cannot extrapolate one token past it.
Absolute sinusoidal	<code>PE(pos, 2i) = sin(pos/10000^(2i/d))</code> <code>PE(pos, 2i+1) = cos(pos/10000^(2i/d))</code>	- Original Transformer encoding using fixed sine/cosine functions at different frequencies, no trainable parameters - provides unique position signals but doesn't explicitly encode relative distances.

Method	Example	Description
ALiBi (Attention with Linear Biases)	<code>attn = m * dist(i, j)</code>	- Subtracts a linear distance penalty from attention scores, with a fixed slope per head - no position embeddings needed - strong extrapolation to longer sequences, but a recency bias that hurts long-range retrieval.
Relative positional encoding	<code>bias = learned_bias[k - q]</code>	- Encodes the distance between positions rather than absolute indices, added to keys and values in attention - better length generalization but computationally expensive for long sequences.
NoPE (No Positional Encoding)	<code>no pos embeddings in some layers</code>	- Injects no explicit positional signal; the causal mask supplies implicit order in decoder-only models - used selectively, e.g. SmolLM3 drops RoPE on every 4th layer - extends beyond explicit schemes but still has a limited context length.

TABLE 3

Model Architecture Variants

The same transformer building blocks can be wired in fundamentally different ways, and that wiring decides what a model is good at. Decoder-only models dominate today because their autoregressive design scales and generates so well, but encoder-only models still win on pure understanding tasks, and encoder-decoder models suit translation. The later rows cover the structural innovations reshaping frontier models: Mixture of Experts for scaling parameters cheaply, reasoning models that think before answering, and vision-language models that fold images into the same context.

Type	Example	Description
Decoder-only	GPT-2, GPT-3, LLaMA, Qwen3	- Causal masked attention for autoregressive generation - trained to predict next token - dominates modern LLMs due to scalability and generation quality.
Encoder-only	BERT, RoBERTa	- Bidirectional attention processes entire sequence - excels at understanding tasks like classification, NER, question answering - cannot generate text autoregressively.
Encoder-decoder	T5, BART	- Separate encoder (bidirectional) and decoder (causal) with cross-attention - optimal for seq2seq tasks like translation and summarization.
Mixture of Experts (MoE)	<code>router(x) → top-k experts</code>	- Sparsely activated architecture where gating network routes tokens to a subset of expert FFNs - scales parameters without proportional compute increase (Mixtral, DeepSeek-V3).
Reasoning model	<code><think>...</think>\nAnswer</code>	- Decoder-only model trained with RL to produce extended chain-of-thought in a scratchpad before the final answer - emergent self-reflection and verification (o1, DeepSeek-R1).
Vision-language	CLIP, LLaVA, Gemini	- Integrates vision encoder (ViT) with language model via projection layer or cross-attention - enables multimodal understanding from images and text jointly.

TABLE 4

Tokenization Algorithms

Before a model ever sees text it has to be chopped into tokens, and the algorithm that does the chopping shapes vocabulary size, how gracefully the model handles rare or unseen words, and even how well it works across languages. These methods mostly differ in how they decide which character or subword pieces to merge or keep: by frequency in BPE, by likelihood in WordPiece, by probability in Unigram, with byte-level variants guaranteeing that any string at all can be encoded without unknown tokens.

Algorithm	Example	Description
Byte-Pair Encoding (BPE)	<code>"playing" → ["play", "ing"]</code>	- Iteratively merges the most frequent adjacent pair in the training corpus - balances vocabulary size with coverage - used in GPT-2 and GPT-3; stores merge rules.

Algorithm	Example	Description
SentencePiece	space encoded as the <code>"_"</code> symbol	- Language-agnostic tokenizer operating on raw text with no pre-tokenization - encodes whitespace as an ordinary symbol - supports BPE and unigram models - used in T5, LLaMA.
WordPiece	"unaffable" → ["un", "##aff", "##able"]	- Similar to BPE but selects merges by likelihood maximization rather than raw frequency - used in BERT - saves the final vocabulary only, not the merge operations.
Unigram language model	probabilistic token selection	- Keeps a vocabulary with per-token probabilities - starts large and removes tokens iteratively to minimize loss - allows multiple segmentations unlike greedy BPE/WordPiece.
Byte-level BPE	any byte sequence tokenizable	- Operates on raw UTF-8 bytes so any string is tokenizable with no unknown tokens - used in GPT-2 and GPT-4 (tiktoken) - fully language-agnostic.

TABLE 5

Pre-training Objectives

The pre-training objective is the self-supervised game a model plays over trillions of tokens, and it determines what kind of model you end up with. Predicting the next token (CLM) gives you a generator like GPT; masking and predicting tokens from both sides (MLM) gives you a bidirectional understander like BERT; corruption-and-reconstruction objectives power text-to-text models like T5. The remaining rows cover newer twists like multi-token prediction for speed and contrastive learning for aligning text with images.

Objective	Example	Description
Causal language modeling (CLM)	$P(\text{token}_i \mid \text{token}_{<i})$	- Predict next token given left context only - standard decoder-only objective maximizing likelihood of training sequences - used in GPT family.
Masked language modeling (MLM)	$P([\text{MASK}] \mid \text{context})$	- Randomly masks ~15% of tokens and predicts them from bidirectional context - BERT's core pre-training objective enabling deep bidirectional representations.
Multi-token prediction (MTP)	predict tokens $t+1, t+2, t+3$	- Trains multiple prediction heads to predict several future tokens simultaneously - improves sample efficiency (4-token prediction up to 3x faster at inference) and the heads can be reused for speculative decoding (DeepSeek-V3).
Denoising autoencoding	corrupt → reconstruct	- Masks or corrupts spans of text then trains model to reconstruct original - T5 frames all tasks as text-to-text generation with varying corruption strategies.
Prefix language modeling	bidirectional prefix → causal	- Applies bidirectional attention to prefix then causal attention for continuation - bridges encoder and decoder benefits (UniLM, GLM).
Contrastive learning	$\text{align}(\text{text}, \text{image})$	- Trains model to match positive pairs (text-image) while separating negative pairs - CLIP uses dual encoders with contrastive loss for vision-language alignment.

TABLE 6

Fine-Tuning Techniques

Once a model is pre-trained, fine-tuning adapts it to a specific task or behavior, and the central tension here is cost versus completeness. Full supervised fine-tuning updates every parameter, but the parameter-efficient methods that follow (LoRA, QLoRA, DoRA, adapters, prefix and prompt tuning) freeze the base model and train a tiny fraction of new weights instead, slashing memory enough to fine-tune huge models on a single GPU while keeping quality close. Instruction tuning sits apart as the step that teaches a model to follow commands rather than just continue text.

Technique	Example	Description
Supervised fine-tuning (SFT)	train on (input, output) pairs	- Standard gradient-based training on task-specific labeled data - updates all parameters to adapt pre-trained model to downstream task.

Technique	Example	Description
Instruction tuning	"Translate: [text]" → output	- Fine-tunes on diverse instruction-following datasets formatted as explicit commands - dramatically improves zero-shot task generalization (FLAN, InstructGPT).
LoRA (Low-Rank Adaptation)	$\Delta W = BA$ (rank $r \ll d$)	- Freezes base model and trains low-rank decomposition matrices injected into attention layers - reduces trainable parameters by 10,000× while preserving quality.
QLoRA	4-bit base + LoRA adapters	- Quantizes frozen base model to 4-bit precision and trains LoRA adapters kept at higher precision - enables fine-tuning 65B models on a single 48GB GPU with minimal degradation.
DoRA (Weight-Decomposed LoRA)	$W = m \cdot (W_0 + BA) / \ W_0 + BA\ $	- Decomposes pre-trained weights into magnitude and direction components - trains direction via LoRA and magnitude separately - consistently outperforms LoRA with no extra inference cost.
Adapter modules	insert bottleneck layers	- Adds small trainable feed-forward modules between frozen transformer layers - modular approach allowing task-specific adapters without full model copies.
Prefix tuning	prepend trainable vectors	- Optimizes continuous task-specific vectors prepended to each layer - keeps model frozen - competitive with full fine-tuning on some tasks with 0.1% parameters.
P-tuning / Prompt tuning	optimize soft prompts	- Learns continuous prompt embeddings rather than discrete text - more parameter-efficient than prefix tuning - effectiveness increases with model scale.

TABLE 7

Alignment and RLHF

Alignment is what turns a raw next-token predictor into a model that is helpful, harmless, and actually does what you ask, and it is one of the fastest-moving areas in the field. The lineage runs from classic three-stage RLHF (reward model plus PPO) toward simpler, cheaper successors: DPO and its variants skip the separate reward model, GRPO drops the critic, and RLVR replaces human labels with programmatic verifiers, which is precisely what unlocked the self-reflecting reasoning models like DeepSeek-R1.

Method	Example	Description
RLHF (Reinforcement Learning from Human Feedback)	reward model → PPO training	- Three-stage process: SFT, train reward model on human preferences, optimize policy via PPO - used in ChatGPT and Claude, computationally expensive.
GRPO (Group Relative Policy Optimization)	sample G outputs → normalize rewards	- Samples a group of responses per prompt and estimates advantages by normalizing rewards within the group - foregoes the critic (value) model, roughly halving memory vs. PPO - used in DeepSeek-R1.
DPO (Direct Preference Optimization)	directly optimize preferences	- Bypasses the reward model by directly optimizing the policy on preference pairs with a simple classification loss (Bradley-Terry) - simpler and more stable than RLHF with comparable results.
SimPO (Simple Preference Optimization)	avg log-prob as implicit reward	- Removes the reference model by using the average log-probability of a response as the implicit reward - adds a target reward margin, outperforming DPO on AlpacaEval 2 with no extra memory cost.
ORPO (Odds Ratio Preference Optimization)	combine SFT + preference in one loss	- Merges SFT and preference alignment into a single training objective using odds ratios - reference model-free, so one pass replaces the separate alignment phase.
KTO (Kahneman-Tversky Optimization)	thumbs-up / thumbs-down labels	- Works with binary feedback (desirable / undesirable) instead of pairwise preferences - cheaper data collection for production systems with like/dislike signals - derived from prospect theory.

Method	Example	Description
RLVR (RL with Verifiable Rewards)	unit test pass / math checker → reward	- Uses programmatic verifiers (unit tests, math checkers) as the reward signal instead of human labels - enables emergent self-reflection and verification for math and code tasks.
DAPO (Decoupled Clip and Dynamic Sampling Policy Optimization)	clip-higher + token-level loss	- Extends GRPO for long chain-of-thought reasoning via Clip-Higher (entropy-collapse prevention), dynamic sampling, token-level policy gradient loss, and overlong reward shaping - reaches 50 on AIME 2024 in 50% of DeepSeek-R1-Zero's training steps.
Constitutional AI	self-critique + revision	- Model critiques its own outputs against constitutional principles then revises, followed by RLAIIF - reduces reliance on human feedback for harmlessness alignment.
RLAIF (RL from AI Feedback)	AI-generated preferences	- Replaces human labelers with AI-generated preferences for reward model training - scales preference data collection - reaches performance comparable to RLHF.
Reward modeling	score(response) from pairs	- Trains a model to predict human preference between response pairs (Bradley-Terry) - outputs a single scalar reward score, converting subjective preferences into a signal for RL optimization.

TABLE 8

Inference Optimization Techniques

Serving an LLM cheaply and fast is its own engineering discipline, and these techniques attack the bottlenecks that make autoregressive generation slow and memory-hungry. KV and prefix caching avoid recomputing work across tokens and requests, Flash Attention and PagedAttention squeeze far more out of GPU memory, and speculative methods like draft models and Medusa generate several tokens at once to break the one-token-at-a-time barrier. Together they're what make production inference economically viable.

Technique	Example	Description
KV caching	cache computed K, V	- Stores previously computed key-value pairs during autoregressive generation - avoids redundant computation—essential for production inference efficiency.
Prefix caching	cache system prompt KV once	- Reuses KV cache for identical prompt prefixes across requests - skips re-computing static content like system prompts - up to 90% cost and 85% latency reduction for long prompts.
Flash Attention	tiling + recomputation	- IO-aware attention algorithm using block-wise computation and kernel fusion - reduces memory bandwidth by 5–20× enabling 2–4× speedup on long sequences.
PagedAttention	block-level memory management	- Manages KV cache in non-contiguous blocks like OS paging - reduces memory fragmentation to near zero and raises serving throughput in vLLM by 2–4×.
Speculative decoding	draft model → verify in parallel	- Small draft model generates candidate tokens that large model verifies in parallel 2–3× speedup without changing output distribution.
Medusa	extra decoding heads predict t+1, t+2	- Adds multiple prediction heads to an LLM and verifies candidates via tree-based attention in parallel 2.2–3.6× speedup without a separate draft model.
Continuous batching	dynamic request batching	Evicts finished sequences and immediately adds new ones— in-flight batching —maximizes GPU utilization vs. static batching.
Quantization (INT8/FP8)	convert FP16 → INT8	- Reduces precision of weights/activations to 8-bit or lower 2–4× memory reduction and speedup with <1% accuracy loss using calibration.
Flash Attention 2/3	improved kernel fusion	- Enhanced IO-aware kernels with lower SRAM usage and better parallelization - Flash-3 up to 2× faster than Flash-2 on H100 with asynchrony optimizations.

TABLE 9

Sampling and Decoding Strategies

A model outputs a probability distribution over the next token; the decoding strategy decides which token actually gets picked, and it's the single biggest lever on whether output feels robotic or creative. Greedy decoding always grabs the top choice and tends toward repetition, while temperature, top-p, top-k, and min-p sampling inject controlled randomness to keep generation lively without going off the rails. Beam and contrastive search take different routes for tasks like translation that reward coherence over surprise.

Strategy	Example	Description
Greedy decoding	<code>argmax(logits)</code>	- Selects highest probability token at each step - deterministic and fast but prone to repetitive, low-quality outputs for creative tasks.
Temperature sampling	<code>logits / T before softmax</code>	- Scales logits by T. Lower T (0.1 to 0.7) sharpens the distribution - higher T (1.0 to 2.0) increases randomness and creativity.
Top-p (nucleus) sampling	<code>cumulative prob >= p=0.9</code>	- Dynamically selects smallest set of tokens whose cumulative probability exceeds threshold p - adapts to the distribution shape and is preferred for open-ended generation.
Top-k sampling	<code>sample from top k=40 tokens</code>	- Restricts sampling to k highest-probability tokens - prevents sampling rare tokens, but a fixed k can be too restrictive or too permissive depending on the distribution.
Min-p sampling	<code>filter tokens < min_p * max_prob</code>	- Removes tokens with probability below min_p fraction of the maximum token probability - keeps coherence better than top-p at high temperature while still allowing creativity.
Beam search	<code>keep top-k sequences</code>	- Maintains k parallel hypotheses and returns the sequence with the highest overall probability - balances quality and diversity but is computationally expensive - common for translation.
Contrastive search	<code>argmax(model_score - a*cos_sim)</code>	- Selects tokens that are probable but distinct from the previous context via a cosine similarity penalty - reduces repetition while maintaining coherence.

TABLE 10

Training Optimizations

Training a large model quickly runs into the hard limits of GPU memory and time, and these techniques are the everyday tricks for staying within them. Mixed precision and gradient checkpointing trade bits and recomputation for room to fit bigger models, gradient accumulation fakes a larger batch size, and the optimizer and schedule choices (AdamW with warmup and cosine decay) are the near universal recipe that keeps large model training from diverging.

Technique	Example	Description
Mixed precision (FP16/BF16)	<code>compute in FP16/BF16, FP32 master weights</code>	- Uses 16-bit floats for the forward/backward math with an FP32 master copy of the weights - reduces memory by ~2× and accelerates training with Tensor Cores. BF16 is preferred for stability since its exponent range matches FP32.
Gradient checkpointing	<code>recompute activations</code>	- Trades compute for memory by recomputing forward activations during the backward pass instead of storing them - enables 2-10× larger models at ~20-30% slowdown.
Gradient accumulation	<code>accumulate over N steps</code>	- Simulates a larger effective batch size by summing gradients across microbatches before one update - runs the microbatches sequentially, so it saves memory but does not speed up training.

Technique	Example	Description
AdamW optimizer	<code>decouple weight decay</code>	- Fixes weight decay in Adam by applying it directly to the weights, not through the loss/gradient - improves generalization - standard optimizer for transformer training.
Learning rate warmup	<code>linear increase 0 → max_lr</code>	- Gradually increases learning rate from near zero over initial steps (typically 1-5% of training) - stabilizes early training of large models and prevents divergence.
Cosine annealing	<code>lr = min + 0.5(max-min)(1+cos)</code>	Decreases learning rate following a cosine curve - smooth decay helps the model converge to flatter minima - usually combined with warmup.
GaLore (Gradient Low-Rank Projection)	<code>project grad to low-rank subspace</code>	- Projects gradients into a low-rank subspace via periodic SVD to shrink optimizer state - keeps full-parameter (full-rank) training, unlike LoRA - no inference overhead.

TABLE 11

Distributed Training Strategies

No single GPU can hold a frontier model, so training is spread across many, and these strategies differ in *what* they split. Data parallelism replicates the model and splits the batch; tensor and pipeline parallelism slice the model itself, within layers or across them; and ZeRO/FSDP shard the optimizer states and parameters to wring out memory. Real large-scale runs combine several of these at once, so understanding what each one partitions is the key to reading any training setup.

Strategy	Example	Description
Data parallelism	<code>replicate model across GPUs</code>	- Each GPU holds full model copy processing different data batches - gradients averaged across devices - simplest parallelism but memory-limited by single GPU.
Tensor parallelism	<code>split layers across GPUs</code>	- Partitions individual layers (e.g., attention heads) within model across devices - requires frequent all-reduce communication - used in Megatron-LM for very large models.
ZeRO (Zero Redundancy Optimizer)	<code>shard optimizer states</code>	- Partitions optimizer states, gradients, parameters across devices with on-demand gathering - ZeRO-3 achieves model parallelism memory efficiency with data parallelism simplicity.
FSDP (Fully Sharded Data Parallel)	<code>PyTorch native ZeRO-3</code>	- PyTorch implementation of ZeRO-3 sharding - automatically manages parameter gathering/scattering - simpler API than DeepSpeed for distributed training.
Pipeline parallelism	<code>layer stages on different GPUs</code>	- Splits model vertically by layers creating pipeline stages - microbatching reduces bubble overhead - GPipe, PipeDream frameworks.
Sequence parallelism	<code>partition along sequence dim</code>	- Splits sequence length across GPUs for memory-constrained layers like LayerNorm - extends tensor parallelism reducing activation memory in long sequences.

TABLE 12

Model Compression Techniques

Compression shrinks a trained model so it runs faster and cheaper, ideally without losing much accuracy. Quantization drops weights to lower precision, distillation trains a small student to imitate a large teacher, and pruning cuts out weights or whole components that contribute little. Each strikes a different balance between how much you save and how much retraining or calibration it costs.

Technique	Example	Description
Post-training quantization	GPTQ, AWQ	- Converts trained model weights to lower precision (INT4/8) without retraining - calibration on a small dataset 3 to 4x compression with minimal accuracy loss.
Knowledge distillation	student learns from teacher	- Trains a small student model to match the outputs or intermediate representations of a large teacher - compresses the model while keeping its capabilities, as in DistilBERT.
Pruning (structured/unstructured)	remove low-magnitude weights	- Removes unimportant weights or entire structured components (heads, layers) - can cut parameters 40 to 60% but usually needs calibration or fine-tuning to recover accuracy.
Quantization-aware training (QAT)	simulate quantization during training	- Inserts fake quantization operations in the forward pass to model precision effects - typically reaches higher accuracy than post-training methods.
Low-rank decomposition	$W \approx UV$ (rank r)	- Approximates weight matrices as a product of low-rank matrices - reduces parameters in linear layers - basis of LoRA and similar PEFT methods.

TABLE 13

Context Window and Long-Context Techniques

A model can only attend over so many tokens at once, and attention's quadratic cost makes simply growing that window expensive, so these techniques extend a model's effective reach in two complementary ways. RAG sidesteps the limit by retrieving relevant text on demand, while RoPE scaling, position interpolation, and YaRN stretch the position encoding to handle far longer sequences than training. Sliding-window and sparse attention cut the quadratic cost so the longer context is actually affordable to run.

Technique	Example	Description
Retrieval-Augmented Generation (RAG)	retrieve docs → augment prompt	- Retrieves relevant documents from external knowledge base and injects into context - extends effective knowledge beyond model limits - requires good retrieval system.
RoPE scaling	adjust rotation frequencies	- Rescales RoPE base frequency (NTK-aware) to extrapolate to longer sequences - simple method enabling 2–4× context extension with minimal fine-tuning.
Position interpolation	compress position indices	Interpolates positions within training range rather than extrapolating - better stability than direct extrapolation for extended context.
YaRN (Yet another RoPE extension)	scale + adjust NTK base	- Combines NTK-aware scaling with attention temperature adjustment per head - efficient context extension from 4K to 128K+ tokens.
Sliding window attention	attend to local window	- Each token attends only to fixed-size window around its position - linear memory but limited long-range modeling - used in Longformer.
Sparse attention	attend to subset of positions	- Computes attention only for selected position pairs using patterns (local, strided, global) - reduces $O(n^2)$ complexity enabling 10×+ longer sequences.
Recurrent memory	compress past into memory	- Summarizes earlier context into compressed memory state - enables unbounded context in theory but loses fine-grained information from distant past.

TABLE 14

Prompt Engineering Techniques

Prompting is how you steer a frozen model at inference time, no training required, and a surprising amount of capability is unlocked just by how you ask. Few-shot and zero-shot prompting set the baseline, while the reasoning-oriented techniques are the real workhorses: chain-of-thought coaxes the model to show its steps, self-consistency votes across multiple attempts, and ReAct and tree-of-thoughts add tool use and search for genuinely hard problems.

Technique	Example	Description
Few-shot prompting	Example1, Example2, ... Query	- Demonstrates task through 2–10 input-output examples in prompt - exploits in-context learning at inference, no weight update - effectiveness grows with model scale - examples should be diverse.
Zero-shot prompting	"Translate to French: [text]"	- Provides task instruction only without examples - relies on pre-training and instruction tuning - quality highly dependent on model capabilities and prompt clarity.
Chain-of-thought (CoT)	"Let's think step by step"	- Prompts model to generate intermediate reasoning steps before final answer - dramatically improves performance on math, logic, commonsense reasoning - an emergent ability that appears mainly at large scale.
Self-consistency	sample multiple paths → vote	- Generates multiple reasoning paths then selects most consistent answer via majority voting - improves reliability over single-path CoT - costs roughly N× the compute for N samples.
ReAct (Reasoning + Acting)	Thought → Action → Observation	- Interleaves reasoning and tool use - model generates thoughts, selects actions (API calls, searches), observes results iteratively until solution found - grounds answers in external data to reduce hallucination.
Tree-of-thoughts	explore reasoning tree	- Explores multiple reasoning branches with self-evaluation, lookahead, and backtracking - enables deliberate problem-solving for complex tasks requiring search.
Skeleton-of-thought	outline → parallel expand	- Generates a skeleton outline first then expands each point in parallel - cuts end-to-end latency (up to ~2×) via parallel API calls or batched decoding - best for list-style answers, not step-by-step math or coding.

TABLE 15

Emergent Capabilities and Scaling

This is the theory behind why making models bigger keeps paying off. Scaling laws predict how loss falls as a power law with compute, data, and size, and the Chinchilla result refined that into how to split a fixed budget, revealing that many earlier models were badly undertrained. The stranger entries are in-context learning and emergent abilities: skills like few-shot learning and step by step reasoning that seem to appear once a model crosses a certain scale, though whether that emergence is real or a measurement artifact is debated.

Concept	Example	Description
Scaling laws	$L(N) \propto N^{(-\alpha)}$	- Loss scales as a power law with compute, model size, and dataset size - smooth and predictable across many orders of magnitude - architecture details like width and depth matter far less than raw scale.
Compute-optimal scaling	Chinchilla scaling	- For a fixed compute budget, balance model size and training tokens - roughly 20 training tokens per parameter is compute-optimal - many earlier models were undertrained relative to their size.
In-context learning	few-shot without gradients	- Learn a task from examples in the prompt with no weight updates - improves with scale - may be driven by induction heads in the attention layers.

Concept	Example	Description
Emergent abilities	reasoning, arithmetic	- Capabilities that appear to show up suddenly at scale, absent in smaller models - examples include multi-step arithmetic and chain-of-thought reasoning - debated: the sharp jumps may be artifacts of the scoring metric.
Transfer learning	pre-train → fine-tune	- Pretrained models encode general language ability that transfers to downstream tasks - fine-tune with far less labeled data than training from scratch - foundation of modern NLP, and larger models tend to transfer better.

TABLE 16

Activation Functions in Transformers

The activation function inside each feed-forward layer is a small choice with a measurable effect on quality. The field has moved from the simple ReLU of the original Transformer toward smoother and gated variants: GELU in BERT and GPT-2, then the gated SwiGLU (LLaMA, PaLM) and GeGLU (T5 v1.1, Gemma), which squeeze out a bit more performance at the cost of a third weight matrix in the block.

Function	Example	Description
SwiGLU	$\text{SwiGLU}(x) = \text{Swish}(xW) \odot (xV)$	- Gated variant using Swish activation ($x \cdot \text{sigmoid}(x)$) with element-wise gating - used in LLaMA, PaLM, and beats GELU at matched parameters - adds a third weight matrix (~50% more FFN parameters at the same hidden size), so the hidden dimension is shrunk to compensate.
GELU (Gaussian Error Linear Unit)	$\text{GELU}(x) = x \cdot \Phi(x)$	- Smooth activation weighting each input by the Gaussian CDF - used in BERT, GPT-2 - better gradient behavior than ReLU, non-monotonic near zero - probabilistic view: the expected value of a stochastic keep-or-zero mask.
GeGLU	$\text{GeGLU}(x) = \text{GELU}(xW) \odot (xV)$	- Like SwiGLU but gates with GELU instead of Swish - top perplexity in the GLU Variants paper, tied with SwiGLU - used in T5 v1.1 and Gemma.
ReLU (Rectified Linear Unit)	$\text{ReLU}(x) = \max(0, x)$	- Simple piecewise linear function - the original Transformer used ReLU - computationally cheap but can suffer dead neurons - largely replaced by gated and smooth variants in modern LLMs.

TABLE 17

Normalization Techniques

Normalization keeps activations in a stable range so deep transformers can train without blowing up, and two questions dominate the design: which normalizer, and where to put it. RMSNorm has largely displaced classic LayerNorm in modern LLMs for being faster at comparable quality, while the Pre-LN versus Post-LN placement decides how stable training is and whether you need a warmup. Dropout, once standard, fades in the largest models that are data-starved rather than overfit.

Method	Example	Description
RMSNorm (Root Mean Square Normalization)	$\text{RMSNorm}(x) = x / \sqrt{\text{RMS}(x) \cdot \gamma}$	- Simplified LN dropping mean centering, normalizes by RMS only while keeping a learned scale - cuts runtime by roughly 7-64% depending on the model, at comparable quality - used in LLaMA, Grok, Qwen3.
Layer Normalization (LN)	$\text{LN}(x) = (x - \mu) / \sigma \cdot \gamma + \beta$	- Normalizes across the feature dimension for each token independently - standard in transformers - mean/variance computed per-sample allowing any batch size, even 1.
Pre-LN vs Post-LN	Pre: $\text{LN}(x) \rightarrow \text{Sublayer}$	Pre-LN normalizes before the sublayer, inside the residual branch (modern default, more stable) Post-LN normalizes after the residual add (original Transformer, needs warmup) - Pre-LN keeps gradients well-behaved at init, enabling training without warmup.

Method	Example	Description
Dropout	randomly zero with $p=0.1$	- Randomly drops units during training as regularization, preventing co-adaptation - often turned off in very large LLMs whose single-epoch pretraining barely overfits - typical rates 0.1-0.2.

TABLE 18

Evaluation Metrics and Benchmarks

Knowing how good a model actually is means picking the right yardstick, and these fall into two camps. Benchmarks like MMLU, GPQA, HumanEval, and SWE-bench probe knowledge, science, and coding ability, though many are nearing saturation or risk contamination, which is why contamination-resistant and human-preference evaluations (LiveBench, Chatbot Arena) have gained ground. The classic automatic metrics (perplexity, BLEU, ROUGE, BERTScore) measure narrower properties like fluency, translation overlap, and summarization quality.

Metric	Example	Description
MMLU (Massive Multitask Language Understanding)	57 subjects, 4-way multiple choice	- Tests knowledge and reasoning across STEM, humanities, social sciences - standard benchmark for general capabilities 0-100% accuracy.
GPQA Diamond	198-question hardest subset of GPQA	- Tests doctoral-level science reasoning in biology, physics, chemistry - the Diamond subset is the hardest slice where experts agree and non-experts fail - designed to be hard even with internet access (Google-proof).
HumanEval	code synthesis benchmark	- Evaluates code generation with 164 hand-written programming problems - pass@k metric measures functional correctness via unit tests - standard for coding models.
SWE-bench	GitHub issue -> code fix	- Tests real-world software engineering by resolving GitHub issues in Python repos - requires edits across multiple files, measures fraction of issues resolved - key agentic coding benchmark.
LiveBench	monthly refreshed questions	- Contamination-limited benchmark with questions updated monthly from recent sources - scored automatically against objective ground truth - covers math, coding, reasoning, language, data analysis.
Arena ELO / Chatbot Arena	human preference pairwise ranking	- Crowdsourced pairwise preference evaluation - relative Elo-style rating from blind human votes - strong signal for real-world conversational quality.
Perplexity	$PPL = \exp(\text{avg_loss})$	- Measures how surprised the model is by test data - lower is better - exponential of average cross-entropy loss - tokenizer-dependent, so not comparable across different tokenizers.
BLEU	n-gram precision with brevity	- Compares n-gram overlap precision between generated and reference translations 0-100 scale - standard for machine translation evaluation.
ROUGE	ROUGE-L, ROUGE-N	- Measures recall-oriented n-gram overlap - primarily for summarization - ROUGE-L uses longest common subsequence.
BERTScore	contextual embedding similarity	- Computes token similarity using contextual embeddings rather than exact matches - captures paraphrase and semantic similarity better than n-gram metrics.

TABLE 19

Attention Mechanism Optimizations

Standard multi-head attention is expensive at inference, mostly because of the memory the KV cache consumes, so these optimizations rework attention to be cheaper. GQA and MQA shrink the cache by sharing keys and values across heads, MLA compresses them into a low-rank latent (the trick behind DeepSeek-V3), and sliding-window, linear, and Flash Attention attack the quadratic cost from different angles, most of them trading a sliver of quality for major savings in speed and memory.

Optimization	Example	Description
Grouped-query attention (GQA)	heads share K, V in groups	• Compromise between MQA and MHA: groups of heads share K, V • balances quality-efficiency tradeoff • used in LLaMA-2/3, Mistral, Qwen3.
Multi-Head Latent Attention (MLA)	compress KV to latent → cache	• Compresses keys and values into a low-rank latent vector (joint KV compression) before caching • reduces KV cache by up to 93% vs. MHA with better modeling quality • used in DeepSeek-V3, Kimi K2.
Multi-query attention (MQA)	single K, V across heads	• Shares same key-value projections across all heads using only multiple queries • reduces KV cache memory and speeds inference but slightly lower quality than GQA.
Flash Attention 2/3	improved kernel fusion	• Enhanced IO-aware kernels with lower SRAM usage and better parallelization • Flash-3 up to 2× faster than Flash-2 on H100 with asynchrony optimizations.
Sliding window attention	attend to k nearest tokens	• Restricts attention to fixed-size local window • reduces complexity to $O(n \cdot k)$ from $O(n^2)$ • enables longer sequences but limited global context.
Linear attention	kernel trick approximation	• Approximates softmax attention using kernel methods reducing complexity to $O(n)$ • enables efficient very long sequences but quality gaps remain vs. full attention.

TABLE 20

Advanced Training Techniques

Beyond the standard recipe, these techniques shape *what* and *in what order* a model learns. Multi-task and curriculum learning structure the training signal — sharing parameters across tasks, or sequencing easy examples before hard ones — while continual learning tackles the problem of updating a model on new data without erasing what it already knew. Contrastive learning and data augmentation round out the toolkit for representation quality and squeezing more from limited data.

Technique	Example	Description
Multi-task learning	train on multiple tasks jointly	• Shares parameters across tasks expecting positive transfer • T5 frames everything as text-to-text • requires balanced sampling and task weighting.
Curriculum learning	easy → hard examples	• Orders training data from simple to complex • can improve convergence and final performance • domain-specific curriculum design needed.
Continual learning	incremental data updates	• Updates model on new data without forgetting previous knowledge • addresses catastrophic forgetting through rehearsal, regularization, or architectural solutions.
Contrastive learning	SimCLR, CLIP	• Learns representations by contrasting positive pairs against negatives • CLIP aligns text-image pairs • effective for self-supervised and multimodal learning.
Data augmentation	backtranslation, paraphrasing	• Generates synthetic training variations from existing data • back-translation, EDA, GPT-generated examples • particularly useful for low-resource tasks.

TABLE 21

Model Merging Techniques

Merging combines two or more fine-tuned models into one without any retraining, blending their abilities purely by manipulating weights, a cheap way to fuse, say, a coding model and a chat model into a single model with no extra inference cost. The methods differ in how they reconcile conflicting weights: SLERP interpolates two models geometrically, task arithmetic adds and subtracts capability "vectors," and TIES and DARE resolve interference when merging many models at once, with evolutionary search automating the recipe hunt.

Technique	Example	Description
SLERP (Spherical Linear Interpolation)	<code>t=0.5</code> between model A and B	- Smoothly interpolates between two models' weights in spherical space preserving geometric properties - best for high-quality pairwise merges - limited to two models at a time.
TIES-Merging	<code>trim</code> → <code>elect sign</code> → <code>disjoint merge</code>	- Three-step process: trim redundant parameters, elect dominant sign direction, merge aligned parameters - handles multi-model merging by resolving parameter conflicts.
DARE (Drop And REscale)	<code>drop delta weights</code> <code>p=0.9, rescale</code>	- Randomly drops task-vector delta weights then rescales remaining by $1/(1-p)$ - effective even dropping 90-99% of deltas - used as augment for TIES or Task Arithmetic.
Task Arithmetic	<code>task_vector =</code> <code>fine_tuned -</code> <code>pretrained</code>	- Computes task vectors (delta weights) and combines via arithmetic - add vectors to merge capabilities, negate to remove behaviors - simple and composable.
Passthrough (layer stacking)	<code>layers 0-32 of A +</code> <code>24-32 of B</code>	- Concatenates layers from different models to create frankenmerge with exotic parameter counts (e.g., 9B from two 7B models) - experimental but can produce capable models.
Evolutionary merging	<code>evolutionary search</code> <code>over merge configs</code>	- Uses evolutionary algorithms to automatically discover optimal merging recipes and hyperparameters - searches both parameter space (weights) and data-flow space (layer path) - enables cross-domain merging without gradient training.
Model Soup (weight averaging)	<code>average weights of N</code> <code>fine-tuned models</code>	Averages weights of multiple fine-tuned versions of same base model - improves accuracy without increasing inference cost - greedy variant evaluates each addition.

TABLE 22

LLM Agent Concepts

An agent is what you get when an LLM can act in the world instead of only talking, and tool use, the ability to call external functions through structured JSON, is the capability that crosses that line. The rest describe how agents string those actions together: the ReAct loop interleaves reasoning with acting, memory spans a single session or persists across many, planning breaks big tasks apart, and multi-agent setups and RLVR-trained agents push toward systems that solve genuinely multi-step problems on their own.

Concept	Example	Description
Tool use / Function calling	<code>{"function":</code> <code>"search", "args":</code> <code>{...}}</code>	- LLM emits a structured JSON call naming an external function and its arguments; your runtime executes it and feeds the result back - the defining capability separating conversational models from agents.
ReAct loop	<code>Thought</code> → <code>Action</code> → <code>Observation</code> → ...	- Agent iterates Thought → Action → Observation cycles until the task is complete - interleaves reasoning and acting so each step is grounded in a real result, cutting hallucination.
Agent memory (short/long-term)	<code>context window +</code> <code>vector store</code>	Short-term: the in-context window, cleared after the session Long-term: an external vector store or database enabling retrieval across sessions (the model has no built-in persistent memory).
Planning and decomposition	<code>task</code> → <code>subtask1,</code> <code>subtask2, ...</code>	- Agent breaks a large task into manageable subtasks via chain-of-thought or tree-of-thought - can reflect and revise the plan based on intermediate results.

Concept	Example	Description
Multi-agent framework	orchestrator → specialist agents	- Multiple LLM agents collaborate: a central orchestrator decides subtasks on the fly and delegates to specialists - improves performance on tasks needing diverse expertise or parallel execution.
Agentic RAG	agent decides when/what to retrieve	- Agent dynamically decides when to retrieve, what to search, and how to use results - contrasts with static one-shot RAG by iterating retrieval based on partial answers.
RLVR for agents	env reward → policy update	- Trains agents using verifiable environment rewards (tool outcomes, test pass/fail) rather than a learned reward model - enables agents to discover multi-step strategies without human demonstrations.

References

LLM papers & model docs

- AdamW optimizer - <https://arxiv.org/abs/1711.05101>
- Agent memory (short/long-term) - <https://www.ibm.com/think/topics/ai-agent-memory>
- ALiBi (Attention with Linear Biases) - <https://arxiv.org/abs/2108.12409>
- BERTScore - <https://arxiv.org/abs/1904.09675>
- Byte-level BPE - <https://huggingface.co/learn/nlp-course/chapter6/5>
- Causal (masked) attention - https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html#decoder
- Chain-of-thought (CoT) - <https://arxiv.org/abs/2201.11903>
- Constitutional AI - <https://arxiv.org/abs/2212.08073>
- Continuous batching - <https://www.anyscale.com/blog/continuous-batching-llm-inference>
- Contrastive learning - <https://arxiv.org/abs/2004.11362>
- Cosine annealing - <https://arxiv.org/abs/1608.03983>
- Curriculum learning - <https://arxiv.org/abs/2101.10382>
- DARE (Drop And REscale) - <https://arxiv.org/abs/2311.03099>
- Data parallelism - https://pytorch.org/tutorials/beginner/ddp_series_theory.html
- Denoising autoencoding - <https://arxiv.org/abs/1910.13461>
- DPO (Direct Preference Optimization) - <https://arxiv.org/abs/2305.18290>
- Emergent abilities - <https://arxiv.org/abs/2206.07682>
- Encoder-only - https://huggingface.co/docs/transformers/model_doc/bert
- Feed-forward network (FFN) - https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html#positionwise-feed-forward-networks
- Flash Attention - <https://arxiv.org/abs/2205.14135>
- FSDP (Fully Sharded Data Parallel) - https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html
- GELU (Gaussian Error Linear Unit) - <https://arxiv.org/abs/1606.08415>
- Gradient accumulation - https://huggingface.co/docs/transformers/perf_train_gpu_one#gradient-accumulation
- Greedy decoding - <https://huggingface.co/blog/how-to-generate>
- GRPO (Group Relative Policy Optimization) - <https://arxiv.org/abs/2402.03300>
- Instruction tuning - <https://arxiv.org/abs/2109.01652>
- KTO (Kahneman-Tversky Optimization) - <https://arxiv.org/abs/2402.01306>
- Layer normalization - <https://arxiv.org/abs/1607.06450>
- Learning rate warmup - <https://arxiv.org/abs/1706.02677>
- LiveBench - <https://arxiv.org/abs/2406.19314>
- Masked language modeling (MLM) - <https://arxiv.org/abs/1810.04805>
- Min-p sampling - <https://arxiv.org/abs/2407.01082>
- Mixture of Experts (MoE) - <https://arxiv.org/abs/1701.06538>
- Adapter modules - <https://arxiv.org/abs/1902.00751>
- Agentic RAG - <https://arxiv.org/html/2501.09136v2>
- Arena ELO / Chatbot Arena - <https://arxiv.org/abs/2403.04132>
- BLEU - <https://aclanthology.org/P02-1040/>
- Byte-Pair Encoding (BPE) - <https://arxiv.org/abs/1508.07909>
- Causal language modeling (CLM) - https://huggingface.co/docs/transformers/tasks/language_modeling
- Compute-optimal scaling - <https://arxiv.org/abs/2203.15556>
- Continual learning - <https://arxiv.org/abs/2302.00487>
- Contrastive learning - <https://arxiv.org/abs/2103.00020>
- Contrastive search - <https://arxiv.org/abs/2210.14140>
- Cross-attention - <https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>
- DAPO (Decoupled Clip and Dynamic Sampling Policy Optimization) - <https://arxiv.org/abs/2503.14476>
- Data augmentation - <https://arxiv.org/abs/2110.01852>
- Decoder-only - https://huggingface.co/docs/transformers/model_doc/gpt2
- DoRA (Weight-Decomposed LoRA) - <https://arxiv.org/abs/2402.09353>
- Dropout - <https://jmlr.org/papers/v15/srivastava14a.html>
- Encoder-decoder - https://huggingface.co/docs/transformers/model_doc/t5
- Evolutionary merging - <https://www.nature.com/articles/s42256-024-00975-8>
- Few-shot prompting - <https://arxiv.org/abs/2005.14165>
- Flash Attention 2/3 - <https://arxiv.org/abs/2307.08691>
- GaLore (Gradient Low-Rank Projection) - <https://arxiv.org/abs/2403.03507>
- GPQA Diamond - <https://arxiv.org/abs/2311.12022>
- Gradient checkpointing - <https://arxiv.org/abs/1604.06174>
- Grouped-query attention (GQA) - <https://arxiv.org/abs/2305.13245>
- HumanEval - <https://arxiv.org/abs/2107.03374>
- Knowledge distillation - <https://arxiv.org/abs/1503.02531>
- KV caching - <https://arxiv.org/abs/2211.05102>
- Learned absolute - <https://huggingface.co/docs/transformers/glossary#position-ids>
- Linear attention - <https://arxiv.org/abs/2006.16236>
- LoRA (Low-Rank Adaptation) - <https://arxiv.org/abs/2106.09685>
- Medusa - <https://arxiv.org/abs/2401.10774>
- Mixed precision (FP16/BF16) - <https://arxiv.org/abs/1710.03740>
- MLLU (Massive Multitask Language Understanding) - <https://arxiv.org/abs/2009.03300>

67. Model Soup (weight averaging) - <https://arxiv.org/abs/2203.05482>
69. Multi-Head Latent Attention (MLA) - <https://arxiv.org/abs/2405.04434>
71. Multi-task learning - <https://arxiv.org/abs/1910.10683>
73. NoPE (No Positional Encoding) - <https://arxiv.org/abs/2404.12224>
75. P-tuning / Prompt tuning - <https://arxiv.org/abs/2103.10385>
77. Perplexity - <https://huggingface.co/docs/transformers/perplexity>
79. Position interpolation - <https://arxiv.org/abs/2306.15595>
81. Pre-LN vs Post-LN - <https://arxiv.org/abs/2002.04745>
83. Prefix language modeling - <https://arxiv.org/abs/1905.03197>
85. Pruning (structured/unstructured) - <https://arxiv.org/abs/2301.00774>
87. QLoRA - <https://arxiv.org/abs/2305.14314>
89. Quantization-aware training (QAT) - <https://arxiv.org/abs/2004.09602>
91. Reasoning model - <https://arxiv.org/abs/2501.12948>
93. Relative positional encoding - <https://arxiv.org/abs/1803.02155>
95. Residual connections - https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html#residual-connections-and-layer-normalization
97. Reward modeling - <https://arxiv.org/abs/2204.05862>
99. RLHF (Reinforcement Learning from Human Feedback) - <https://arxiv.org/abs/2203.02155>
101. RoPE (Rotary Position Embedding) - <https://arxiv.org/abs/2104.09864>
103. ROUGE - <https://aclanthology.org/W04-1013/>
105. Self-attention - <https://jalammar.github.io/illustrated-transformer/>
107. SentencePiece - <https://arxiv.org/abs/1808.06226>
109. SimPO (Simple Preference Optimization) - <https://arxiv.org/abs/2405.14734>
111. SLERP (Spherical Linear Interpolation) - <https://huggingface.co/blog/mlabonne/merge-models>
113. Sparse attention - <https://arxiv.org/abs/1904.10509>
115. Supervised fine-tuning (SFT) - https://huggingface.co/docs/trl/sft_trainer
117. SwiGLU - <https://arxiv.org/abs/2002.05202>
119. Tensor parallelism - <https://arxiv.org/abs/1909.08053>
121. Top-k sampling - <https://arxiv.org/abs/1805.04833>
123. Transfer learning - <https://arxiv.org/abs/1801.06146>
125. Unigram language model - <https://arxiv.org/abs/1804.10959>
127. WordPiece - <https://arxiv.org/abs/1609.08144>
129. ZeRO (Zero Redundancy Optimizer) - <https://arxiv.org/abs/1910.02054>
68. Multi-head attention - https://d2l.ai/chapter_attention-mechanisms-and-transformers/multihead-attention.html
70. Multi-query attention (MQA) - <https://arxiv.org/abs/1911.02150>
72. Multi-token prediction (MTP) - <https://arxiv.org/abs/2404.19737>
74. ORPO (Odds Ratio Preference Optimization) - <https://arxiv.org/abs/2403.07691>
76. PagedAttention - <https://arxiv.org/abs/2309.06180>
78. Pipeline parallelism - <https://arxiv.org/abs/1811.06965>
80. Positional encoding - https://kazemnejad.com/blog/transformer_architecture_positional_encoding/
82. Prefix caching - <https://bentoml.com/llm/inference-optimization/prefix-caching>
84. Prefix tuning - <https://arxiv.org/abs/2101.00190>
86. QK-Norm - <https://arxiv.org/abs/2302.05442>
88. Quantization (INT8/FP8) - <https://arxiv.org/abs/2208.07339>
90. ReAct (Reasoning + Acting) - <https://arxiv.org/abs/2210.03629>
92. Recurrent memory - <https://arxiv.org/abs/2404.07143>
94. ReLU (Rectified Linear Unit) - <https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html>
96. Retrieval-Augmented Generation (RAG) - <https://arxiv.org/abs/2005.11401>
98. RLAIF (RL from AI Feedback) - <https://arxiv.org/abs/2309.00267>
100. RMSNorm (Root Mean Square Normalization) - <https://arxiv.org/abs/1910.07467>
102. RoPE scaling - https://huggingface.co/docs/transformers/main/en/internal/rope_utils
104. Scaling laws - <https://arxiv.org/abs/2001.08361>
106. Self-consistency - <https://arxiv.org/abs/2203.11171>
108. Sequence parallelism - <https://arxiv.org/abs/2205.05198>
110. Skeleton-of-thought - <https://arxiv.org/abs/2307.15337>
112. Sliding window attention - <https://arxiv.org/abs/2004.05150>
114. Speculative decoding - <https://arxiv.org/abs/2211.17192>
116. SWE-bench - <https://arxiv.org/abs/2310.06770>
118. Task Arithmetic - <https://arxiv.org/abs/2212.04089>
120. TIES-Merging - <https://arxiv.org/abs/2306.01708>
122. Top-p (nucleus) sampling - <https://arxiv.org/abs/1904.09751>
124. Tree-of-thoughts - <https://arxiv.org/abs/2305.10601>
126. Vision-language - <https://huggingface.co/blog/vlms>
128. YaRN (Yet another RoPE extension) - <https://arxiv.org/abs/2309.00071>

Vendor documentation & tools

130. Planning and decomposition - <https://www.anthropic.com/engineering/building-effective-agents>
131. Tool use / Function calling - <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>